

MAPTR: 面向在线向量化高精地图构建的结构化建模与学习 (STRUCTURED MODELING AND LEARNING FOR ONLINE VECTORIZED HD MAP CONSTRUCTION)

Bencheng Liao^{1, 2, 3*} Shaoyu Chen^{2, 3*} Xinggang Wang^{2†}
Tianheng Cheng^{2, 3} Qian Zhang³ Wenyu Liu² Chang Huang³

¹ Institute of Artificial Intelligence, Huazhong University of Science & Technology

² School of EIC, Huazhong University of Science & Technology

³ Horizon Robotics

{bcliao, shaoyuchen, xgwang, thch, liuwu}@hust.edu.cn

{qian01.zhang, chang.huang}@horizon.ai

ABSTRACT

高精地图 (High-definition Map, HD Map) 提供了驾驶场景丰富且精确的环境信息, 是自动驾驶 (Autonomous Driving) 系统中规划模块的基础性和不可或缺的组成部分。本文提出 MapTR——一种面向高效在线向量化高精地图构建 (Online Vectorized HD Map Construction) 的结构化端到端 (End-to-End) Transformer。本文提出统一的排列等价建模 (Permutation-equivalent Modeling) 方法, 即将地图元素 (Map Element) 建模为带有等价排列 (Equivalent Permutations) 集合的点集 (Point Set), 该方法准确描述地图元素形状并稳定学习过程。本文设计层次化查询嵌入 (Hierarchical Query Embedding) 方案以灵活编码结构化地图信息, 并执行层次化二分匹配 (Hierarchical Bipartite Matching) 进行地图元素学习。在 nuScenes 数据集上, MapTR 仅凭相机输入即在现有向量化地图构建方法中达到最优性能和效率。具体而言, MapTR-nano 在 RTX 3090 上实现实时推理速度 (25.1 FPS), 比现有基于相机的当前最优 (State-of-the-Art, SOTA) 方法快 8 倍, 同时 mAP 高 5.0。即使与现有当前最优多模态 (Multi-modality) 方法相比, MapTR-nano 的 mAP 高 0.7, MapTR-tiny 的 mAP 高 13.5 且推理速度快 3 倍。丰富的定性结果表明, MapTR 在复杂多样的驾驶场景中保持稳定鲁棒的地图构建质量。MapTR 在自动驾驶中具有重要的应用价值。代码和更多演示见 <https://github.com/hustvl/MapTR>。

1 引言

高精地图是针对自动驾驶专门设计的高精度地图, 由地图元素的实例级向量化表示组成, 包括人行横道 (Pedestrian Crossing)、车道线 (Lane Divider)、道路边界 (Road Boundary) 等。高精地图包含道路拓扑和交通规则丰富语义信息, 对自动驾驶车辆导航至关重要。

传统上, 高精地图通过基于 SLAM 的方法 (Zhang & Singh, 2014; Shan & Englot, 2018; Shan et al., 2020) 进行离线构建, 流程复杂且维护成本高。近年来, 在线高精地图构建受到越来越多关注, 该方法利用车载传感器在运行过程中实时构建自车周围地图, 摆脱了离线人工操作。

早期工作 (Chen et al., 2022a; Liu et al., 2021a; Can et al., 2021) 利用线形先验从前视图图像中感知开放形状的车道线。这类方法局限于单视图感知 (Perception), 无法处理其他任意形状的地面元素。随着鸟瞰视图 (Bird’s Eye View, BEV) 表示学习的发展, 近期工作 (Chen et al., 2022b; Zhou & Krähenbühl, 2022; Hu et al., 2021; Li et al., 2022c) 通过 BEV 语义分割预测栅格化地图 (Rasterized Map)。然而, 栅格化地图缺乏向量化实例级信息, 如车道结构等, 这对下游任务 (Downstream Tasks) (如运动预测和规划) 至关重要。为构建向量化高精地图, HDMapNet (Li et al., 2022a) 对逐像素分割结果进行分组, 这需要复杂且耗

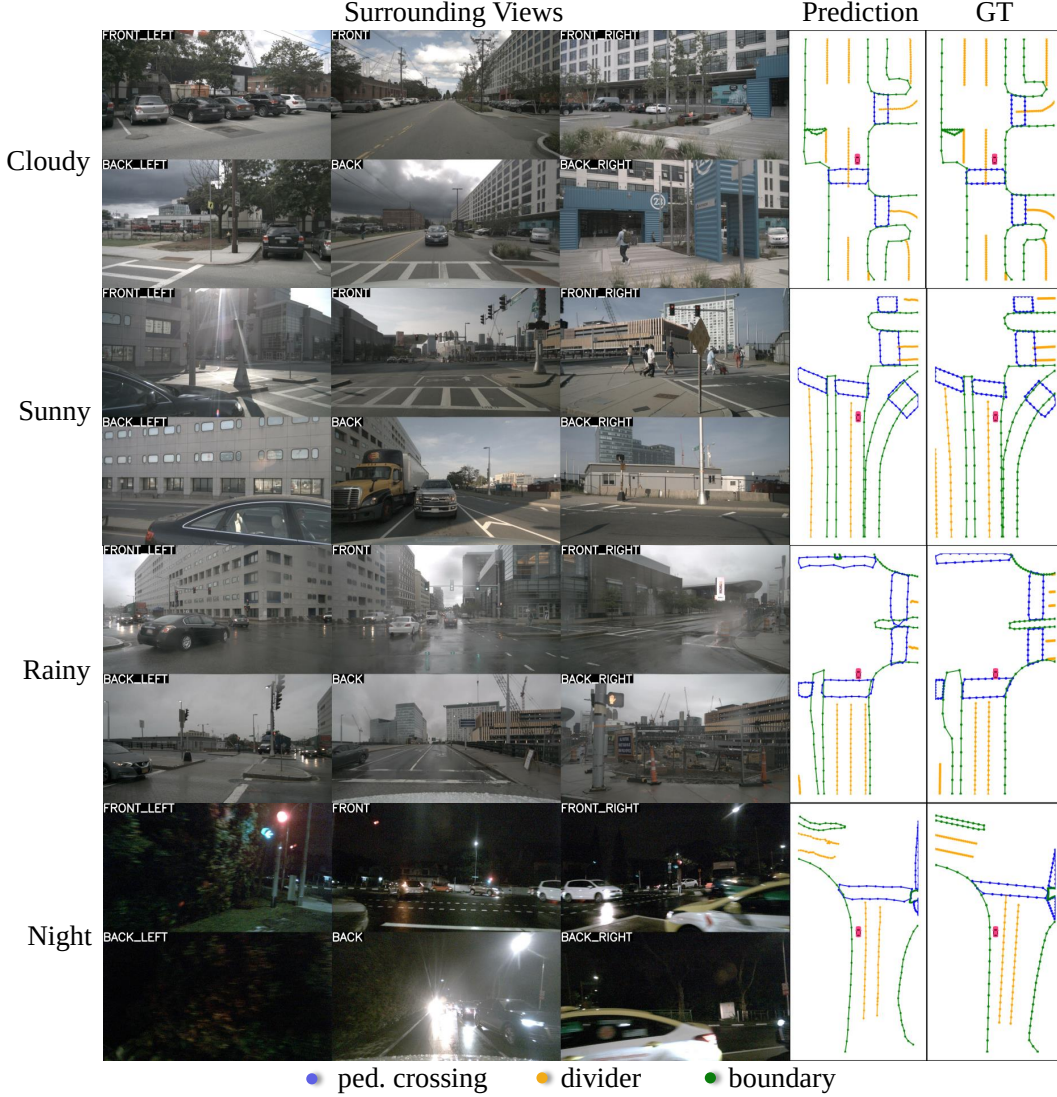


图 1. MapTR 在复杂多样的驾驶场景中保持稳定鲁棒的向量化高精地图构建质量。时的后处理 (Post-processing)。VectorMapNet (Liu et al., 2022a) 将每个地图元素表示为点序列 (Point Sequence)，采用级联的从粗到细 (Coarse-to-fine) 框架，利用自回归解码器 (Auto-regressive Decoder) 逐点预测，导致推理时间较长。

现有在线向量化高精地图构建方法受限于效率，无法应用于实时场景。最近，DETR (Carion et al., 2020) 采用简洁高效的编码器-解码器 (Encoder-Decoder) Transformer 架构，实现了端到端目标检测 (Object Detection)。

这自然引出一个问题：能否设计类似 *DETR* 的范式来实现高效的端到端向量化高精地图构建？本文通过提出的 **Map TR**ansformer (MapTR) 表明答案是肯定的。

与目标检测中可将目标简单抽象为边界框 (Bounding Box) 不同，向量化地图元素具有更动态的形状。为精确描述地图元素，本文提出统一建模方法。将每个地图元素建模为带有等价排列集合的点集。点集确定地图元素位置，排列组包含与该几何形状对应的所有可能点集组织顺序，从而避免形状模糊性。

基于排列等价建模，本文设计结构化框架，以车载相机图像为输入，输出向量化高精地图。将在线向量化高精地图构建简化为并行回归 (Parallel Regression) 问题。提出层次化查询嵌入以灵活编码实例级和点级信息。所有实例及其所有点通过统一 Transformer 结构同时预测。训练流程被归结为层次化集合预测 (Set Prediction) 任务，依次执行层次化二分匹配以分配实例和点。通过提出的点对点损失 (Point2point Loss) 和边缘方向损失 (Edge Direction Loss)，在点级和边缘级监督几何形状。

综合以上全部设计，本文提出 MapTR——一种采用统一建模和架构的高效端到端在线向量化高精地图构建方法。MapTR 在 nuScenes (Caesar et al., 2020) 数据集上达到现有向量化地图构建方法中最佳性能和效率。具体而言，MapTR-nano 在 RTX 3090 上实现实时推理速度 (25.1 FPS)，比现有基于相机的当前最优方法快 8 倍，同时 mAP 高 5.0。即使与现有当前最优多模态方法相比，MapTR-nano 的 mAP 高 0.7 且推理速度快 8 倍，MapTR-tiny 的 mAP 高 13.5 且推理速度快 3 倍。如图 1 所示，MapTR 在复杂多样的驾驶场景中保持稳定鲁棒的地图构建质量。

本文贡献总结如下：

- 提出统一的排列等价建模方法用于地图元素，即将地图元素建模为带有等价排列集合的点集，该方法准确描述地图元素形状并稳定学习过程。
- 基于该建模方式，提出 MapTR——一种面向高效在线向量化高精地图构建的结构化端到端框架。设计层次化查询嵌入方案以灵活编码实例级和点级信息，执行层次化二分匹配进行地图元素学习，通过提出的点对点损失和边缘方向损失在点级和边缘级监督几何形状。
- MapTR 是首个实时且当前最优 (SOTA) 的向量化高精地图构建方法，在复杂多样的驾驶场景中具有稳定鲁棒的性能。

2 相关工作

高精地图构建 (HD Map Construction) . 近年来，随着 2D 到 BEV 方法 (Ma et al., 2022) 的发展，高精地图构建被归结为基于车载相机环视图像数据的分割问题。Chen et al. (2022b); Zhou & Krähenbühl (2022); Hu et al. (2021); Li et al. (2022c); Phillion & Fidler (2020); Liu et al. (2022b) 通过 BEV 语义分割生成栅格化地图。为构建向量化高精地图，HDMapNet (Li et al., 2022a) 通过启发式且耗时的后处理对逐像素语义分割结果进行分组以生成实例。VectorMapNet (Liu et al., 2022a) 作为首个端到端框架，采用两阶段从粗到细框架并利用自回归解码器逐点预测，导致推理时间长且存在排列模糊性。与 VectorMapNet 不同，MapTR 引入统一的地图元素建模方式，解决了模糊性并稳定了学习过程。MapTR 构建了结构化的并行单阶段框架，效率大幅提升。

车道线检测 (Lane Detection) . 车道线检测 (Lane Detection) 可视作高精地图构建的子任务，专注于检测道路场景中的车道元素。由于大多数车道线检测数据集仅提供单视图标注且聚焦于开放形状元素，相关方法局限于单视图。LaneATT (Tabelini et al., 2021) 利用基于锚点的深度车道线检测模型，在准确率和效率之间取得良好平衡。LSTR (Liu et al., 2021a) 采用 Transformer 架构直接输出车道形状模型参数。GANet (Wang et al., 2022) 将车道线检测归结为关键点估计和关联问题，采用自底向上的设计方案。Feng et al. (2022) 提出基于参数化贝塞尔曲线 (Bezier Curve) 的车道线检测方法。Garnett et al. (2019) 不直接在 2D 图像坐标中检测车道，而是提出 3D-LaneNet 在 BEV 中进行 3D 车道线检测。STSU (Can et al., 2021) 将车道线表示为 BEV 坐标中的有向图，采用基于曲线的贝塞尔方法从单目相机图像预测车道线。Persformer (Chen et al., 2022a) 提供更好的 BEV 特征表示并优化锚点设计，同时统一 2D 和 3D 车道线检测。MapTR 不局限于在有限单视图中检测车道线，而是通过统一建模和学习框架感知 360° 水平视野内的各类地图元素。

基于轮廓的实例分割 (Contour-based Instance Segmentation) . 与 MapTR 相关的另一条研究路线是基于轮廓 (Contour) 的 2D 实例分割 (Zhu et al., 2022; Xie et al., 2020; Xu et al., 2019; Liu et al., 2021c)。这些方法将 2D 实例分割重新定义为目标轮廓预测任务，估计轮廓顶点的图像坐标。CurveGCN (Ling et al., 2019) 利用图卷积网络 (Graph Convolution Network) 预测多边形边界。Lazarow et al. (2022); Liang et al. (2020); Li et al. (2021); Peng et al. (2020) 依赖中间表示并采用两阶段范式，即第一阶段执行分割/检测生成顶点，第二阶段将顶点转换为多边形。这些工作将 2D 实例掩码的轮廓建模为多边形，其建模方法无法处理线形地图元素，不适用于地图构建。相比之下，MapTR 专为高精地图构建设计，以统一方式建模各类地图元素。此外，MapTR 不依赖中间表示，流程高效紧凑。

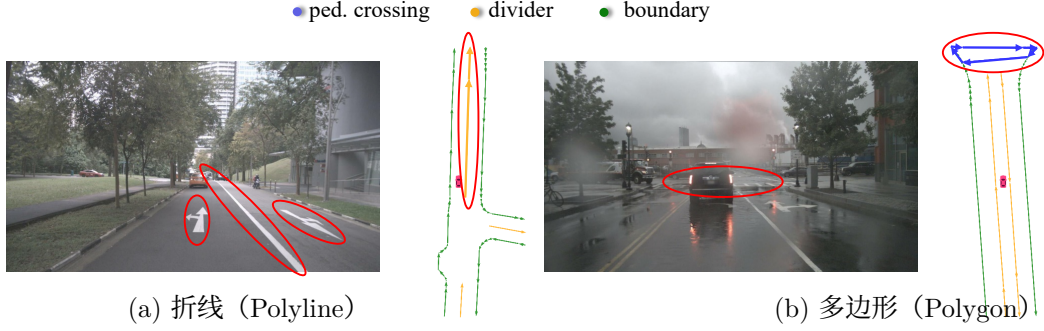


图 2. 说明地图元素关于起点和方向模糊性的典型案例。(a) 折线：对于两条对向车道之间的车道线，定义其方向较为困难。车道线的两个端点均可视为起点，点集可按两个方向组织。(b) 多边形：对于人行横道，多边形的每个点均可视为起点，多边形可按两个相反方向（逆时针和顺时针）连接。

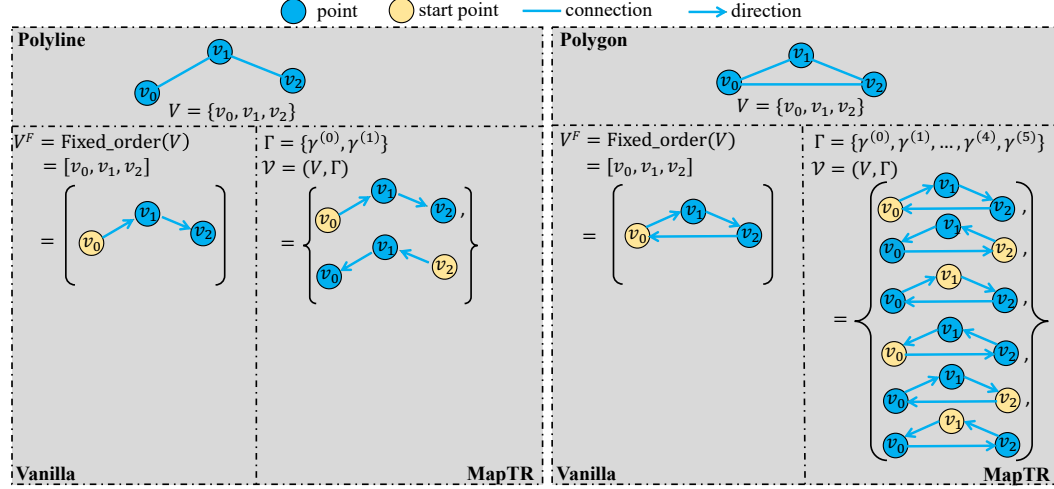


图 3. MapTR 排列等价建模示意图。地图元素被几何抽象并离散为折线和多边形。MapTR 使用 (V, Γ) (点集 V 和等价排列组 Γ) 建模每个地图元素，避免了模糊性并稳定了学习过程。

3 MAPTR

3.1 排列等价建模 (PERMUTATION-EQUIVALENT MODELING)

MapTR 旨在以统一方式建模和学习高精地图。高精地图是向量化静态地图元素的集合，包括人行横道、车道线、道路边界等。在结构化建模中，MapTR 将地图元素几何抽象为闭合形状（如人行横道）和开放形状（如车道线）。通过沿形状边界顺序采样点，闭合形状元素离散为多边形 (Polygon)，开放形状元素离散为折线 (Polyline)。

初步地，多边形和折线均可表示为有序点集 $V^F = [v_0, v_1, \dots, v_{N_v-1}]$ (见图 3 (Vanilla))。 N_v 表示点的数量。然而，点集的排列并未明确定义且不唯一。多边形和折线存在许多等价排列。例如，如图 2 (a) 所示，对于两条对向车道之间的车道线（折线），定义其方向较为困难。车道线的两个端点均可视为起点，点集可按两个方向组织。在图 2 (b) 中，对于人行横道（多边形），点集可按两个相反方向（逆时针和顺时针）组织，且循环改变点集的排列对多边形的几何形状无影响。将固定排列强加于点集作为监督是不合理的。强加的固定排列与其他等价排列相矛盾，阻碍了学习过程。

为弥补这一差距，MapTR 使用 $\mathcal{V} = (V, \Gamma)$ 建模每个地图元素。 $V = \{v_j\}_{j=0}^{N_v-1}$ 表示地图元素的点集 (N_v 为点数)。 $\Gamma = \{\gamma^k\}$ 表示点集 V 的等价排列组，涵盖所有可能的组织顺序。

Specifically, for polyline element (see Fig. 3 (left)), Γ includes 2 kinds of equivalent permutations:

$$\Gamma_{\text{polyline}} = \{\gamma^0, \gamma^1\} \begin{cases} \gamma^0(j) = j \mod N_v, \\ \gamma^1(j) = (N_v - 1) - j \mod N_v. \end{cases} \quad (1)$$

For polygon element (see Fig. 3 (right)), Γ includes $2 \times N_v$ kinds of equivalent permutations:

$$\Gamma_{\text{polygon}} = \{\gamma^0, \dots, \gamma^{2 \times N_v - 1}\} \left\{ \begin{array}{l} \gamma^0(j) = j \mod N_v, \\ \gamma^1(j) = (N_v - 1) - j \mod N_v, \\ \gamma^2(j) = (j + 1) \mod N_v, \\ \gamma^3(j) = (N_v - 1) - (j + 1) \mod N_v, \\ \dots \\ \gamma^{2 \times N_v - 2}(j) = (j + N_v - 1) \mod N_v, \\ \gamma^{2 \times N_v - 1}(j) = (N_v - 1) - (j + N_v - 1) \mod N_v. \end{array} \right. \quad (2)$$

通过引入等价排列概念, MapTR 以统一方式建模地图元素, 解决了模糊性问题。MapTR 进一步引入层次化二分匹配 (见第 3.2 节和第 3.3 节) 用于地图元素学习, 并设计结构化编码器-解码器 Transformer 架构以高效预测地图元素 (见第 3.4 节)。

3.2 层次化匹配 (HIERARCHICAL MATCHING)

MapTR 遵循 DETR (Carion et al., 2020; Fang et al., 2021) 的端到端范式, 单次并行推断固定大小的 N 个地图元素集合。 N 设置为大于场景中典型地图元素数量。将 N 个预测地图元素的集合记为 $\hat{Y} = \{\hat{y}_i\}_{i=0}^{N-1}$ 。真实 (GT) 地图元素集合用 $\emptyset$ (无目标) 填充至大小 N , 记为 $Y = \{y_i\}_{i=0}^{N-1}$ 。 $y_i = (c_i, V_i, \Gamma_i)$, 其中 c_i 、 V_i 和 Γ_i 分别为 GT 地图元素 y_i 的目标类别标签、点集和排列组。 $\hat{y}_i = (\hat{p}_i, \hat{V}_i)$, 其中 $\hat{p}_i$ 和 $\hat{V}_i$ 分别为预测分类得分和预测点集。为实现结构化地图元素建模和学习, MapTR 引入层次化二分匹配, 即依次执行实例级匹配和点级匹配。

实例级匹配 (Instance-level Matching) . 首先, 需要在预测地图元素 $\{\hat{y}_i\}$ 和 GT 地图元素 $\{y_i\}$ 之间找到最优的实例级标签分配 $\hat{\pi}$ 。 $\hat{\pi}$ 是 N 个元素的排列 ($\hat{\pi} \in \Pi_N$), 具有最低的实例级匹配代价:

$$\hat{\pi} = \arg \min_{\pi \in \Pi_N} \sum_{i=0}^{N-1} \mathcal{L}_{\text{ins_match}}(\hat{y}_{\pi(i)}, y_i). \quad (3)$$

$\mathcal{L}_{\text{ins_match}}(\hat{y}_{\pi(i)}, y_i)$ 是预测 $\hat{y}_{\pi(i)}$ 与 GT y_i 之间的成对匹配代价, 同时考虑地图元素的类别标签和点集位置:

$$\mathcal{L}_{\text{ins_match}}(\hat{y}_{\pi(i)}, y_i) = \mathcal{L}_{\text{Focal}}(\hat{p}_{\pi(i)}, c_i) + \mathcal{L}_{\text{position}}(\hat{V}_{\pi(i)}, V_i). \quad (4)$$

$\mathcal{L}_{\text{Focal}}(\hat{p}_{\pi(i)}, c_i)$ 是类别匹配代价项, 定义为预测分类得分 $\hat{p}_{\pi(i)}$ 与目标类别标签 c_i 之间的 Focal Loss (Lin et al., 2017)。 $\mathcal{L}_{\text{position}}(\hat{V}_{\pi(i)}, V_i)$ 是位置匹配代价项, 反映预测点集 $\hat{V}_{\pi(i)}$ 与 GT 点集 V_i 之间的位置相关性 (详见第 B 节)。采用匈牙利算法 (Hungarian Algorithm) 寻找最优的实例级分配 $\hat{\pi}$, 遵循 DETR。

点级匹配 (Point-level Matching) . 完成实例级匹配后, 每个预测地图元素 $\hat{y}_{\hat{\pi}(i)}$ 被分配一个 GT 地图元素 y_i 。然后, 对每个分配正标签的预测实例 ($c_i \neq \emptyset$), 执行点级匹配以在预测点集 $\hat{V}_{\hat{\pi}(i)}$ 和 GT 点集 V_i 之间找到最优的点对点分配 $\hat{\gamma} \in \Gamma$ 。 $\hat{\gamma}$ 从预定义排列组 Γ 中选出, 具有最低的点级匹配代价:

$$\hat{\gamma} = \arg \min_{\gamma \in \Gamma} \sum_{j=0}^{N_v-1} D_{\text{Manhattan}}(\hat{v}_j, v_{\gamma(j)}). \quad (5)$$

$D_{\text{Manhattan}}(\hat{v}_j, v_{\gamma(j)})$ 是预测点集 $\hat{V}$ 的第 j 个点与 GT 点集 V 的第 $\gamma(j)$ 个点之间的曼哈顿距离 (Manhattan Distance)。

3.3 训练损失 (TRAINING LOSS)

MapTR 基于最优的实例级和点级分配 ($\hat{\pi}$ 和 $\{\hat{\gamma}_i\}$) 进行训练。损失函数由三部分组成: 分类损失、点对点损失和边缘方向损失:

$$\mathcal{L} = \lambda \mathcal{L}_{\text{cls}} + \alpha \mathcal{L}_{\text{p2p}} + \beta \mathcal{L}_{\text{dir}}, \quad (6)$$

其中 λ 、 α 和 β 为平衡不同损失项的权重。

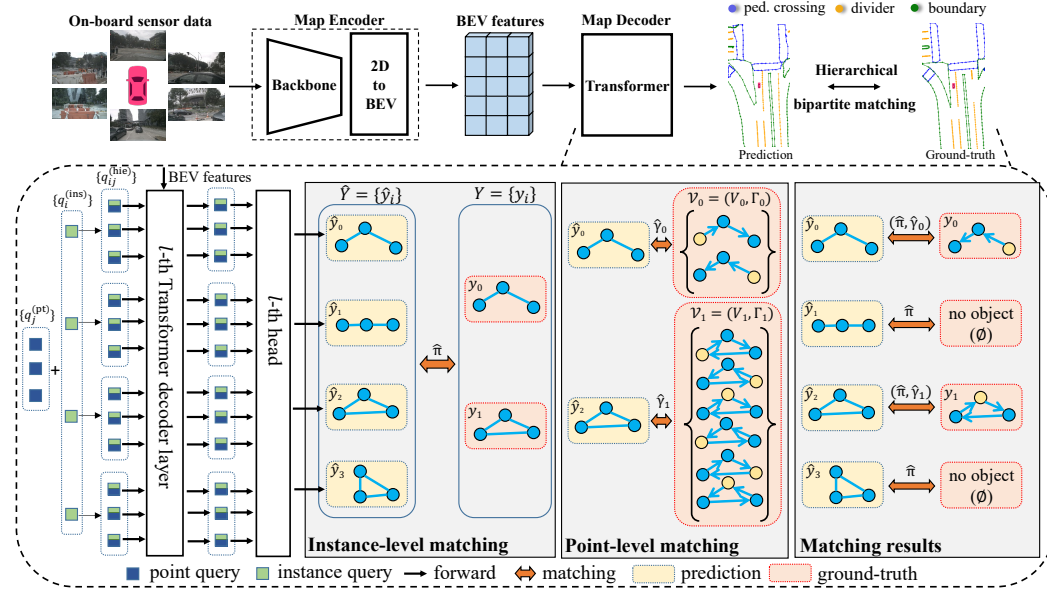


图 4. MapTR 整体架构。MapTR 采用编码器-解码器范式。地图编码器将传感器输入转换为统一的 BEV 表示。地图解码器采用层次化查询嵌入方案显式编码地图元素，并基于排列等价建模执行层次化匹配。MapTR 完全端到端，流程高度结构化、紧凑且高效。

分类损失 (Classification Loss) . 基于实例级最优匹配结果 $\hat{\pi}$ ，每个预测地图元素被分配一个类别标签。分类损失是 Focal Loss 项，公式化为：

$$\mathcal{L}_{cls} = \sum_{i=0}^{N-1} \mathcal{L}_{Focal}(\hat{p}_{\hat{\pi}(i)}, c_i). \quad (7)$$

点对点损失 (Point2point Loss) . 点对点损失监督每个预测点的位置。对于索引为 i 的每个 GT 实例，根据点级最优匹配结果 $\hat{\gamma}_i$ ，每个预测点 $\hat{v}_{\hat{\pi}(i),j}$ 被分配一个 GT 点 $v_{i,\hat{\gamma}_i(j)}$ 。点对点损失定义为每个分配点对之间的曼哈顿距离：

$$\mathcal{L}_{p2p} = \sum_{i=0}^{N-1} \mathbb{1}_{\{c_i \neq \emptyset\}} \sum_{j=0}^{N_v-1} D_{Manhattan}(\hat{v}_{\hat{\pi}(i),j}, v_{i,\hat{\gamma}_i(j)}). \quad (8)$$

边缘方向损失 (Edge Direction Loss) . 点对点损失仅监督折线和多边形的节点点，未考虑边（相邻点之间的连接线）。为精确表示地图元素，边方向很重要。因此，进一步设计边缘方向损失以在更高边缘级监督几何形状。具体而言，考虑成对的预测边 $\hat{e}_{\hat{\pi}(i),j}$ 和 GT 边 $e_{i,\hat{\gamma}_i(j)}$ 的余弦相似度 (Cosine Similarity)：

$$\begin{aligned} \mathcal{L}_{dir} = & - \sum_{i=0}^{N-1} \mathbb{1}_{\{c_i \neq \emptyset\}} \sum_{j=0}^{N_v-1} \text{cosine_similarity}(\hat{e}_{\hat{\pi}(i),j}, e_{i,\hat{\gamma}_i(j)}), \\ & \hat{e}_{\hat{\pi}(i),j} = \hat{v}_{\hat{\pi}(i),j} - \hat{v}_{\hat{\pi}(i),(j+1) \bmod N_v}, \\ & e_{i,\hat{\gamma}_i(j)} = v_{i,\hat{\gamma}_i(j)} - v_{i,\hat{\gamma}_i(j+1) \bmod N_v}. \end{aligned} \quad (9)$$

3.4 网络架构 (ARCHITECTURE)

MapTR 采用编码器-解码器范式，整体架构如图 4 所示。

输入模态 (Input Modality) . MapTR 以车载相机环视图像为输入。MapTR 也兼容其他车载传感器（如激光雷达 (LiDAR) 和雷达 (RADAR)）。将 MapTR 扩展到多模态数据

直接且简单。得益于合理的排列等价建模，即使仅使用相机输入，MapTR 也显著优于使用多模态输入的其他方法。

地图编码器 (Map Encoder) . MapTR 的地图编码器从多台车载相机图像中提取特征，并将这些特征转换为统一的特征表示，即 BEV 表示。给定多视图图像 $\mathcal{I} = \{I_1, \dots, I_K\}$ ，使用常规骨干网络 (Backbone) 生成多视图特征图 $\mathcal{F} = \{F_1, \dots, F_K\}$ 。然后将 2D 图像特征 $\mathcal{F}$ 转换为 BEV 特征 $\mathcal{B} \in \mathbb{R}^{H \times W \times C}$ 。默认采用 GKT (Chen et al., 2022b) 作为基本 2D 到 BEV 转换模块，因其易于部署且高效。MapTR 兼容其他转换方法并保持稳定性能，例如 CVT (Zhou & Krähenbühl, 2022)、LSS (Phillion & Fidler, 2020; Liu et al., 2022c; Li et al., 2022b; Huang et al., 2021)、Deformable Attention (Li et al., 2022c; Zhu et al., 2021) 和 IPM (Mallot et al., 1991)。消融实验见表 4。

地图解码器 (Map Decoder) . 本文提出层次化查询嵌入方案以显式编码每个地图元素。具体而言，定义一组实例级查询 $\{q_i^{\text{ins}}\}_{i=0}^{N_v-1}$ 和一组所有实例共享的点点级查询 $\{q_j^{\text{pt}}\}_{j=0}^{N_v-1}$ 。每个地图元素 (索引 i) 对应一组层次化查询 $\{q_{ij}^{\text{hie}}\}_{j=0}^{N_v-1}$ 。第 i 个地图元素第 j 个点的层次化查询公式化为：

$$q_{ij}^{\text{hie}} = q_i^{\text{ins}} + q_j^{\text{pt}}. \quad (10)$$

地图解码器包含多个级联的解码器层，迭代更新层次化查询。在每个解码器层中，采用 MHSA 使层次化查询相互交换信息 (包括实例间和实例内)。然后采用 Deformable Attention (Zhu et al., 2021) 使层次化查询与 BEV 特征交互，受 BEVFormer (Li et al., 2022c) 启发。每个查询 q_{ij}^{hie} 预测参考点 p_{ij} 的二维归一化 BEV 坐标 (x_{ij}, y_{ij}) 。然后在参考点周围采样 BEV 特征并更新查询。

地图元素通常具有不规则形状且需要长距离上下文。每个地图元素对应一组分布灵活动态的参考点 $\{p_{ij}\}_{j=0}^{N_v-1}$ 。参考点 $\{p_{ij}\}_{j=0}^{N_v-1}$ 可适应地图元素的任意形状，并为地图元素学习捕获信息丰富的上下文。

MapTR 的预测头简单，由分类分支和点回归分支组成。分类分支预测实例类别得分。点回归分支预测点集 $\hat{V}$ 的位置。对于每个地图元素，输出 $2N_v$ 维向量，表示 N_v 个点的归一化 BEV 坐标。

4 实验

数据集与指标 (Dataset and Metric) . 在 nuScenes (Caesar et al., 2020) 数据集上评估 MapTR，该数据集包含 1000 个场景，每个场景约 20 秒。关键样本以 2Hz 频率标注。每个样本包含来自 6 个相机的 RGB 图像，覆盖自车 360° 水平视野。遵循先前方法 (Li et al., 2022a; Liu et al., 2022a)，选择三种地图元素进行公平评估：人行横道、车道线和道路边界。感知范围为 X 轴 $[-15.0m, 15.0m]$ 和 Y 轴 $[-30.0m, 30.0m]$ 。采用平均精度 (Average Precision, AP) 评估地图构建质量。使用 Chamfer 距离 D_{Chamfer} 判断预测与 GT 是否匹配。在多个 D_{Chamfer} 阈值 ($\tau \in T, T = \{0.5, 1.0, 1.5\}$) 下计算 AP_τ ，然后对所有阈值取平均作为最终 AP 指标：

$$\text{AP} = \frac{1}{|T|} \sum_{\tau \in T} \text{AP}_\tau. \quad (11)$$

实现细节 (Implementation Details) . MapTR 使用 8 张 NVIDIA GeForce RTX 3090 GPU 进行训练。采用 AdamW (Loshchilov & Hutter, 2019) 优化器和余弦退火 (Cosine Annealing) 调度。MapTR-tiny 采用 ResNet50 (He et al., 2016) 作为骨干网络，总批量大小为 32 (包含 6 视图图像)。所有消融实验基于 MapTR-tiny 训练 24 个 epoch。MapTR-nano 专为实时应用设计，采用 ResNet18 作为骨干网络。更多细节见附录 A。

4.1 与当前最优方法的比较 (COMPARISONS WITH STATE-OF-THE-ART METHODS)

在表 1 中，将 MapTR 与当前最优方法进行比较。MapTR-nano 在 RTX 3090 上运行达到实时推理速度 (25.1 FPS)，比现有基于相机的当前最优方法 (VectorMapNet-C) 快 8 倍，同

时 mAP 高 5.0。即使与现有当前最优多模态方法相比，MapTR-nano 的 mAP 高 0.7 且推理速度快 8 倍，MapTR-tiny 的 mAP 高 13.5 且推理速度快 3 倍。MapTR 也是快速收敛的方法，在 24 个 epoch 的训练计划下即展现出先进性能。

Method	Modality	Backbone	Epochs	AP_{ped}	$AP_{divider}$	$AP_{boundary}$	mAP	FPS
HMapNet	C	Effi-B0	30	14.4	21.7	33.0	23.0	0.8
HMapNet	L	PointPillars	30	10.4	24.1	37.9	24.1	1.0
HMapNet	C & L	Effi-B0 & PointPillars	30	16.3	29.6	46.7	31.0	0.5
VectorMapNet	C	R50	110	36.1	47.3	39.3	40.9	2.9
VectorMapNet	L	PointPillars	110	25.7	37.6	38.6	34.0	-
VectorMapNet	C & L	R50 & PointPillars	110	37.6	50.5	47.5	45.2	-
MapTR-nano	C	R18	110	39.6	49.9	48.2	45.9	25.1
MapTR-tiny	C	R50	24	46.3	51.5	53.1	50.3	11.2
MapTR-tiny	C	R50	110	56.2	59.8	60.1	58.7	11.2

表 1. 与 nuScenes val 集上当前最优方法 (Liu et al., 2022a; Li et al., 2022a) 的比较。“C”和“L”分别表示相机 (Camera) 和激光雷达 (LiDAR)。“Effi-B0”和“PointPillars”分别对应于 Tan & Le (2019) 和 Lang et al. (2019)。其他方法的 AP 值取自 VectorMapNet 论文。VectorMapNet-C 的 FPS 由其作者提供并在 RTX 3090 上测量。其他 FPS 在同一台 RTX 3090 机器上测量。“-”表示相应结果不可用。即使仅使用相机输入，MapTR-tiny 显著优于多模态方法 (+13.5 mAP)。MapTR-nano 达到基于相机的 SOTA 性能，运行速度为 25.1 FPS，首次实现了实时向量化地图构建。

4.2 消融实验 (ABLATION STUDY)

为验证不同设计的有效性，在 nuScenes val 集上进行消融实验。更多消融实验见附录 B。

排列等价建模的有效性 (Effectiveness of Permutation-equivalent Modeling). 在表 2 中提供了消融实验以验证所提排列等价建模的有效性。与对点集强加固定排列的朴素建模方法相比，排列等价建模解决了地图元素的模糊性，带来了 5.9 mAP 的提升。对于人行横道，提升甚至达到 11.9 AP，证明了在多边形元素建模上的优越性。还在图 5 中可视化了学习过程，展示了所提建模的稳定性。

Modeling method	AP_{ped}	$AP_{divider}$	$AP_{boundary}$	mAP
Fixed-order V^F w/ ambiguity	34.4	48.1	50.7	44.4
Permutation-equivalent (V, Γ) w/o ambiguity	46.3	51.5	53.1	50.3

表 2. 建模方法的消融实验。朴素建模方法对点集强加固定排列，导致模糊性。MapTR 引入排列等价建模以避免模糊性，稳定了学习过程并显著提升性能 (+5.9 mAP)。

边缘方向损失的有效性 (Effectiveness of Edge Direction Loss). 关于边缘方向损失权重的消融实验见表 3。 $\beta = 0$ 表示不使用边缘方向损失。 $\beta = 5e^{-3}$ 对应适当的监督强度，被采用为默认设置。

2D 到 BEV 转换 (2D-to-BEV Transformation). 在表 4 中，对 2D 到 BEV 转换方法进行了消融实验 (如 IPM (Mallot et al., 1991)、LSS (Liu et al., 2022c; Phillion & Fidler, 2020)、Deformable Attention (Li et al., 2022c) 和 GKT (Chen et al., 2022b))。使用 LSS (Liu et al., 2022c) 的优化实现。为与 IPM 和 LSS 公平比较，GKT 和 Deformable Attention 均采用单层配置。实验表明 MapTR 兼容多种 2D 到 BEV 方法并保持稳定性能。采用 GKT 作为 MapTR 的默认配置，因其易于部署且高效。

4.3 定性可视化 (QUALITATIVE VISUALIZATION)

图 1 展示了复杂多样驾驶场景的预测向量化高精地图结果。MapTR 保持稳定且令人印象深刻的结果。更多定性结果见附录 C。还提供了视频 (补充材料中) 以展示鲁棒性。

β	AP_{ped}	$AP_{divider}$	$AP_{boundary}$	mAP
0	41.4	51.3	51.9	48.2
$3e^{-3}$	44.8	50.4	52.1	49.1
$5e^{-3}$	46.3	51.5	53.1	50.3
$1e^{-2}$	41.9	50.9	52.0	48.3

表 3. 关于边缘方向损失权重 β 的消融实验。

Method	mAP	FPS	Param.
IPM	46.2	11.7	35.7M
LSS	49.5	10.0	37.1M
Deform. Atten.	49.7	11.2	36.0M
GKT	50.3	11.2	35.9M

表 4. 2D 到 BEV 转换方法的消融实验。MapTR 兼容多种 2D 到 BEV 方法并保持稳定性能。

5 结论

MapTR 是一种面向高效在线向量化高精地图构建的结构化端到端框架, 采用简洁的编码器-解码器 Transformer 架构和层次化二分匹配, 基于提出的排列等价建模进行地图元素学习。大量实验表明, 所提方法能够在具有挑战性的 nuScenes 数据集上精确感知任意形状的地图元素。期望 MapTR 能够作为自动驾驶系统的基本模块, 推动下游任务(如运动预测和规划)的发展。

致谢

本工作部分受国家自然科学基金(NSFC No. 62276108)资助。感谢刘亦诚在评估方面的指导和建设性讨论。

参考文献

- Holger Caesar, Varun Bankiti, Alex H Lang, Sourabh Vora, Venice Erin Liong, Qiang Xu, Anush Krishnan, Yu Pan, Giancarlo Baldan, and Oscar Beijbom. nusenes: A multimodal dataset for autonomous driving. In *CVPR*, 2020.
- Yigit Baran Can, Alexander Liniger, Danda Pani Paudel, and Luc Van Gool. Structured bird’s-eye-view traffic scene understanding from onboard images. In *ICCV*, 2021.
- Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *ECCV*, 2020.
- Li Chen, Chonghao Sima, Yang Li, Zehan Zheng, Jiajie Xu, Xiangwei Geng, Hongyang Li, Conghui He, Jianping Shi, Yu Qiao, and Junchi Yan. Persformer: 3d lane detection via perspective transformer and the openlane benchmark. In *ECCV*, 2022a.
- Shaoyu Chen, Tianheng Cheng, Xinggang Wang, Wenming Meng, Qian Zhang, and Wenyu Liu. Efficient and robust 2d-to-bev representation learning via geometry-guided kernel transformer. *arXiv preprint arXiv:2206.04584*, 2022b.
- Yuxin Fang, Bencheng Liao, Xinggang Wang, Jiemin Fang, Jiyang Qi, Rui Wu, Jianwei Niu, and Wenyu Liu. You only look at one sequence: Rethinking transformer in vision through object detection. *Advances in Neural Information Processing Systems*, 34:26183–26197, 2021.
- Zhengyang Feng, Shaohua Guo, Xin Tan, Ke Xu, Min Wang, and Lizhuang Ma. Rethinking efficient lane detection via curve modeling. In *CVPR*, 2022.
- Noa Garnett, Rafi Cohen, Tomer Pe’er, Roei Lahav, and Dan Levi. 3d-lanenet: end-to-end 3d multiple lane detection. In *ICCV*, 2019.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016.
- Anthony Hu, Zak Murez, Nikhil Mohan, Sofia Dudas, Jeffrey Hawke, Vijay Badrinarayanan, Roberto Cipolla, and Alex Kendall. FIERY: Future instance segmentation in bird’s-eye view from surround monocular cameras. In *ICCV*, 2021.
- Junjie Huang, Guan Huang, Zheng Zhu, Ye Yun, and Dalong Du. Bevdet: High-performance multi-camera 3d object detection in bird-eye-view. *arXiv preprint arXiv:2112.11790*, 2021.
- Alex H. Lang, Sourabh Vora, Holger Caesar, Lubing Zhou, Jiong Yang, and Oscar Beijbom. Pointpillars: Fast encoders for object detection from point clouds. In *CVPR*, 2019.
- Justin Lazarow, Weijian Xu, and Zhuowen Tu. Instance segmentation with mask-supervised polygonal boundary transformers. In *CVPR*, 2022.
- Qi Li, Yue Wang, Yilun Wang, and Hang Zhao. Hdmapnet: An online hd map construction and evaluation framework. In *ICRA*, 2022a.

- Weijia Li, Wenqian Zhao, Huaping Zhong, Conghui He, and Dahua Lin. Joint semantic-geometric learning for polygonal building segmentation. In *AAAI*, 2021.
- Yinhao Li, Zheng Ge, Guanyi Yu, Jinrong Yang, Zengran Wang, Yukang Shi, Jianjian Sun, and Zeming Li. Bevdepth: Acquisition of reliable depth for multi-view 3d object detection. *arXiv preprint arXiv:2206.10092*, 2022b.
- Zhiqi Li, Wenhao Wang, Hongyang Li, Enze Xie, Chonghao Sima, Tong Lu, Yu Qiao, and Jifeng Dai. Bevformer: Learning bird’s-eye-view representation from multi-camera images via spatiotemporal transformers. In *ECCV*, 2022c.
- Justin Liang, Namdar Homayounfar, Wei-Chiu Ma, Yuwen Xiong, Rui Hu, and Raquel Urtasun. Polytransform: Deep polygon transformer for instance segmentation. In *CVPR*, 2020.
- Tsung-Yi Lin, Priya Goyal, Ross B. Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *ICCV*, 2017.
- Huan Ling, Jun Gao, Amlan Kar, Wenzheng Chen, and Sanja Fidler. Fast interactive object annotation with curve-gcn. In *CVPR*, 2019.
- Ruijin Liu, Zejian Yuan, Tie Liu, and Zhiliang Xiong. End-to-end lane shape prediction with transformers. In *WACV*, 2021a.
- Yicheng Liu, Yue Wang, Yilun Wang, and Hang Zhao. Vectormapnet: End-to-end vectorized hd map learning. *arXiv preprint arXiv:2206.08920*, 2022a.
- Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 10012–10022, 2021b.
- Zhi Liu, Shaoyu Chen, Xiaojie Guo, Xinggang Wang, Tianheng Cheng, Hongmei Zhu, Qian Zhang, Wenyu Liu, and Yi Zhang. Vision-based uneven bev representation learning with polar rasterization and surface estimation. *arXiv preprint arXiv:2207.01878*, 2022b.
- Zhijian Liu, Haotian Tang, Alexander Amini, Xingyu Yang, Huizi Mao, Daniela Rus, and Song Han. Bevfusion: Multi-task multi-sensor fusion with unified bird’s-eye view representation. *arXiv preprint arXiv:2205.13542*, 2022c.
- Zichen Liu, Jun Hao Liew, Xiangyu Chen, and Jiashi Feng. Dance: A deep attentive contour model for efficient instance segmentation. In *WACVW*, 2021c.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.
- Yuexin Ma, Tai Wang, Xuyang Bai, Huitong Yang, Yuenan Hou, Yaming Wang, Y. Qiao, Ruigang Yang, Dinesh Manocha, and Xinge Zhu. Vision-centric bev perception: A survey. *arXiv preprint arXiv:2208.02797*, 2022.
- Hanspeter A Mallot, Heinrich H Bülthoff, JJ Little, and Stefan Bohrer. Inverse perspective mapping simplifies optical flow computation and obstacle detection. *Biological cybernetics*, 1991.
- Sida Peng, Wen Jiang, Huaijin Pi, Xiuli Li, Hujun Bao, and Xiaowei Zhou. Deep snake for real-time instance segmentation. In *CVPR*, 2020.
- Jonah Philion and Sanja Fidler. Lift, splat, shoot: Encoding images from arbitrary camera rigs by implicitly unprojecting to 3d. In *ECCV*, 2020.
- Tixiao Shan and Brendan Englot. Lego-loam: Lightweight and ground-optimized lidar odometry and mapping on variable terrain. In *IROS*, 2018.
- Tixiao Shan, Brendan J. Englot, Drew Meyers, Wei Wang, Carlo Ratti, and Daniela Rus. LIO-SAM: tightly-coupled lidar inertial odometry via smoothing and mapping. In *IROS*, 2020.

- Lucas Tabelini, Rodrigo Berriel, Thiago M Paixao, Claudine Badue, Alberto F De Souza, and Thiago Oliveira-Santos. Keep your eyes on the lane: Real-time attention-guided lane detection. In *CVPR*, 2021.
- Mingxing Tan and Quoc V. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *ICML*, 2019.
- Jinsheng Wang, Yinchao Ma, Shaofei Huang, Tianrui Hui, Fei Wang, Chen Qian, and Tianzhu Zhang. A keypoint-based global association network for lane detection. In *CVPR*, 2022.
- Enze Xie, Peize Sun, Xiaoge Song, Wenhai Wang, Xuebo Liu, Ding Liang, Chunhua Shen, and Ping Luo. Polarmask: Single shot instance segmentation with polar representation. In *CVPR*, 2020.
- Wenqiang Xu, Haiyang Wang, Fubo Qi, and Cewu Lu. Explicit shape encoding for real-time instance segmentation. In *ICCV*, 2019.
- Ji Zhang and Sanjiv Singh. LOAM: lidar odometry and mapping in real-time. In *Robotics: Science and Systems X, University of California*, 2014.
- Brady Zhou and Philipp Krähenbühl. Cross-view transformers for real-time map-view semantic segmentation. In *CVPR*, 2022.
- Chenming Zhu, Xuanye Zhang, Yanran Li, Liangdong Qiu, Kai Han, and Xiaoguang Han. Sharpcontour: A contour-based boundary refinement approach for efficient and accurate instance segmentation. In *CVPR*, 2022.
- Xizhou Zhu, Weijie Su, Lewei Lu, Bin Li, Xiaogang Wang, and Jifeng Dai. Deformable DETR: deformable transformers for end-to-end object detection. In *ICLR*, 2021.

附录

A 实现细节

本节提供方法和实验的更多实现细节。

数据增强 (Data Augmentation). 源图像分辨率为 1600×900 。对于 MapTR-nano, 将源图像按 0.2 比例缩放。对于 MapTR-tiny, 将源图像按 0.5 比例缩放。默认使用颜色抖动 (Color Jitter)。

模型设置 (Model Setting). 所有实验中, 训练时 λ 设为 2, α 设为 5, β 设为 $5e^{-3}$ 。对于 MapTR-tiny, 实例级查询数和点级查询数分别设为 50 和 20。将每个 BEV 网格大小设为 $0.3m$, 堆叠 6 个 Transformer 解码器层。MapTR-tiny 的总批量大小为 32 (包含 6 视图图像), 学习率为 $6e^{-4}$, 骨干网络的学习率乘子为 0.1。所有消融实验基于 MapTR-tiny 训练 24 个 epoch。对于 MapTR-nano, 实例级查询数和点级查询数分别设为 100 和 20。将每个 BEV 网格大小设为 $0.75m$, 堆叠 2 个 Transformer 解码器层。MapTR-nano 训练 110 个 epoch, 总批量大小为 192, 学习率为 $4e^{-3}$, 骨干网络的学习率乘子为 0.1。采用 GKT (Chen et al., 2022b) 作为 MapTR 的默认 2D 到 BEV 模块。

数据集预处理 (Dataset Preprocessing). 按照 Liu et al. (2022a); Li et al. (2022a) 处理地图标注。提取自车感知范围内的地图元素作为 GT 地图元素。默认感知范围为 X 轴 $[-15.0m, 15.0m]$ 和 Y 轴 $[-30.0m, 30.0m]$ 。

B 消融实验

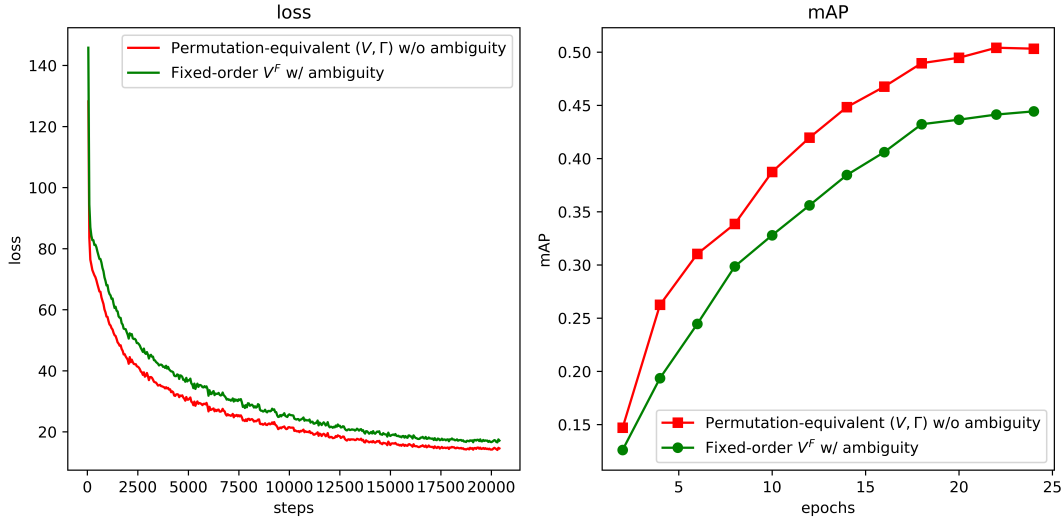


图 5. 排列建模方法的收敛曲线。

点数 (Point Number). 关于每个地图元素建模点数的消融实验见表 5。点数过少无法描述地图元素的复杂几何形状, 点数过多影响效率。默认采用 20 个点。

Pt. num.	AP_{ped}	$AP_{divider}$	$AP_{boundary}$	mAP	FPS
10	42.5	51.3	50.1	48.0	12.3
20	46.3	51.5	53.1	50.3	11.2
40	44.7	52.4	52.9	50.0	10.8

表 5. 关于每个地图元素建模点数的消融实验。

元素数量 (Element Number). 关于地图元素数量的消融实验见表 6。MapTR-tiny 默认采用 50 个地图元素。

Ele. num.	AP _{ped}	AP _{divider}	AP _{boundary}	mAP	FPS
25	36.3	43.8	44.7	41.6	11.4
50	46.3	51.5	53.1	50.3	11.2
75	48.2	53.1	55.3	52.2	11.1

表 6. 关于地图元素数量的消融实验。

解码器层数 (Decoder Layer Number). 关于地图解码器层数的消融实验见表 7。地图构建性能随层数增加而提升, 但在层数达到 6 后趋于饱和。

Layer num.	AP _{ped}	AP _{divider}	AP _{boundary}	mAP	FPS
1	20.8	30.2	36.3	29.1	15.2
2	36.0	43.1	48.0	42.4	14.2
3	38.2	44.1	49.5	44.0	13.5
6	46.3	51.5	53.1	50.3	11.2
8	39.6	51.9	51.2	47.6	10.6

表 7. 关于解码器层数的消融实验。

位置匹配代价 (Position Matching Cost). 如第 3.2 节所述, 在实例级匹配中采用位置匹配代价项 $\mathcal{L}_{\text{position}}(\hat{V}_{\pi(i)}, V_i)$ 以反映预测点集 $\hat{V}_{\pi(i)}$ 与 GT 点集 V_i 之间的位置相关性。在表 8 中比较了两种代价设计, 即 Chamfer 距离代价和点对点代价。点对点代价类似于点级匹配代价, 具体而言, 找到最佳点对点分配, 将所有点对的曼哈顿距离之和作为两个点集的位置匹配代价。实验表明点对点代价优于 Chamfer 距离代价。

Position matching cost	AP _{ped}	AP _{divider}	AP _{boundary}	mAP
Chamfer distance cost	40.3	53.8	48.5	47.5
Point2point cost	46.3	51.5	53.1	50.3

表 8. 关于位置匹配代价项的消融实验。

Swin Transformer 骨干网络 (Swin Transformer Backbones). 关于 Swin Transformer 骨干网络 (Liu et al., 2021b) 的消融实验见表 9。

Method	Backbone	AP _{ped}	AP _{divider}	AP _{boundary}	mAP	FPS	Param.
MapTR-tiny	R50	46.3	51.5	53.1	50.3	11.2	35.9M
MapTR-tiny	Swin-tiny	45.2	52.7	52.3	50.1	9.1	39.9M
MapTR-small	Swin-small	50.2	55.4	57.3	54.3	7.3	61.2M
MapTR-base	Swin-base	50.6	58.7	58.4	55.9	6.1	99.2M

表 9. 关于 Swin Transformer 骨干网络的消融实验。

模态 (Modality). 多传感器感知对自动驾驶车辆的安全至关重要, MapTR 兼容其他车载传感器如激光雷达。如表 10 所示, 在仅 24 个 epoch 的训练计划下, 多模态 MapTR 以 2 倍速度优势显著超越此前当前最优结果 17.3 mAP。

Method	Modality	Epochs	AP _{ped}	AP _{divider}	AP _{boundary}	mAP	FPS
HMapNet	C & L	30	16.3	29.6	46.7	31.0	0.5
VectorMapNet	C & L	110	37.6	50.5	47.5	45.2	<2.9
MapTR-tiny	C	24	46.3	51.5	53.1	50.3	11.2
MapTR-tiny	L	24	48.5	53.7	64.7	55.6	7.2
MapTR-tiny	C & L	24	55.9	62.3	69.3	62.5	5.8

表 10. 关于模态的消融实验。

相机偏差鲁棒性 (Robustness to Camera Deviation). 实际应用中, 相机内参通常准确且变化小, 但相机外参可能因相机位置偏移、标定误差等而不准确。为验证鲁棒性, 遍历验证集并对每个样本随机生成噪声。分别添加不同程度的平移和旋转偏差。注意对所有相机和所有坐标添加噪声, 噪声服从正态分布。部分样本存在极大偏差, 对性能影响较大。如表 11 和表 12 所示, 当 $\Delta_x, \Delta_y, \Delta_z$ 的标准差为 $0.1m$ 或 $\theta_x, \theta_y, \theta_z$ 的标准差为 $0.02rad$ 时, MapTR 仍保持相当性能。

Method	$\sigma_1(m)$				
	0	0.05	0.1	0.5	1.0
MapTR-tiny	50.3	49.9	49.0	34.0	17.0

表 11. 相机平移偏差鲁棒性。指标为 mAP。 σ_1 为 $\Delta_x, \Delta_y, \Delta_z$ 的标准差。

Method	$\sigma_2(rad)$				
	0	0.005	0.01	0.02	0.05
MapTR-tiny	50.3	49.4	47.5	42.0	24.7

表 12. 相机旋转偏差鲁棒性。指标为 mAP。 σ_2 为 $\theta_x, \theta_y, \theta_z$ 的标准差。

详细运行时间 (Detailed Running Time) . 为深入理解 MapTR 的效率, 在表 13 中展示了 MapTR-tiny 仅使用多相机输入时各组件的详细运行时间。

Component	Runtime (ms)	Proportion
Backbone	55.5	62.1%
2D-to-BEV module (GKT)	12.3	13.8%
Map decoder	21.5	24.1%
Total	89.3	100 %

表 13. MapTR-tiny 在 RTX 3090 上各组件的详细运行时间。

C 定性可视化

在 nuScenes val 集上展示了 MapTR 在各种天气条件和具有挑战性的道路环境下的地图构建结果。如图 6、图 7 和图 8 所示, MapTR 保持稳定且令人印象深刻的结果。视频结果见补充材料。

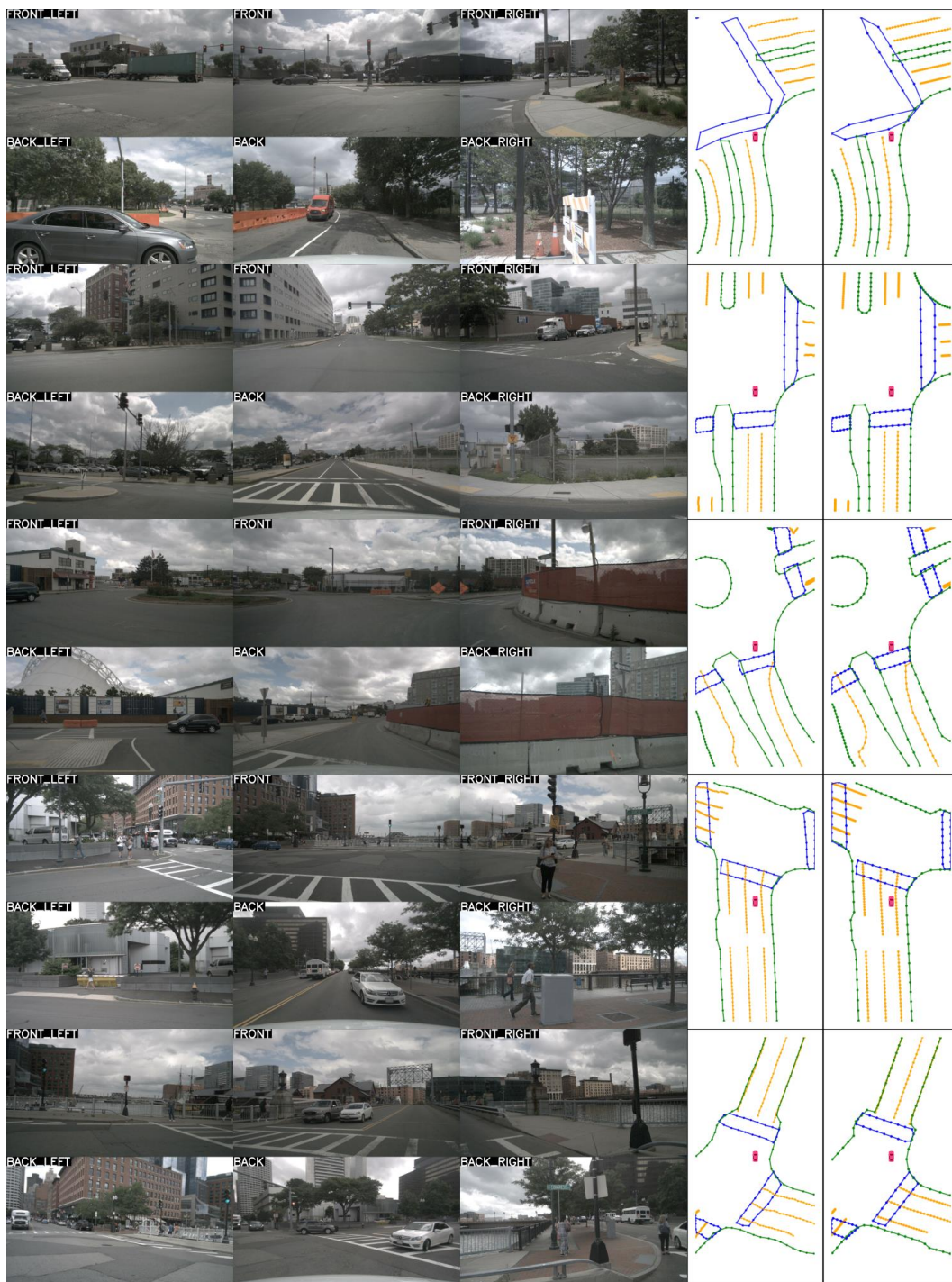


图 6. 晴阴天气下的可视化。

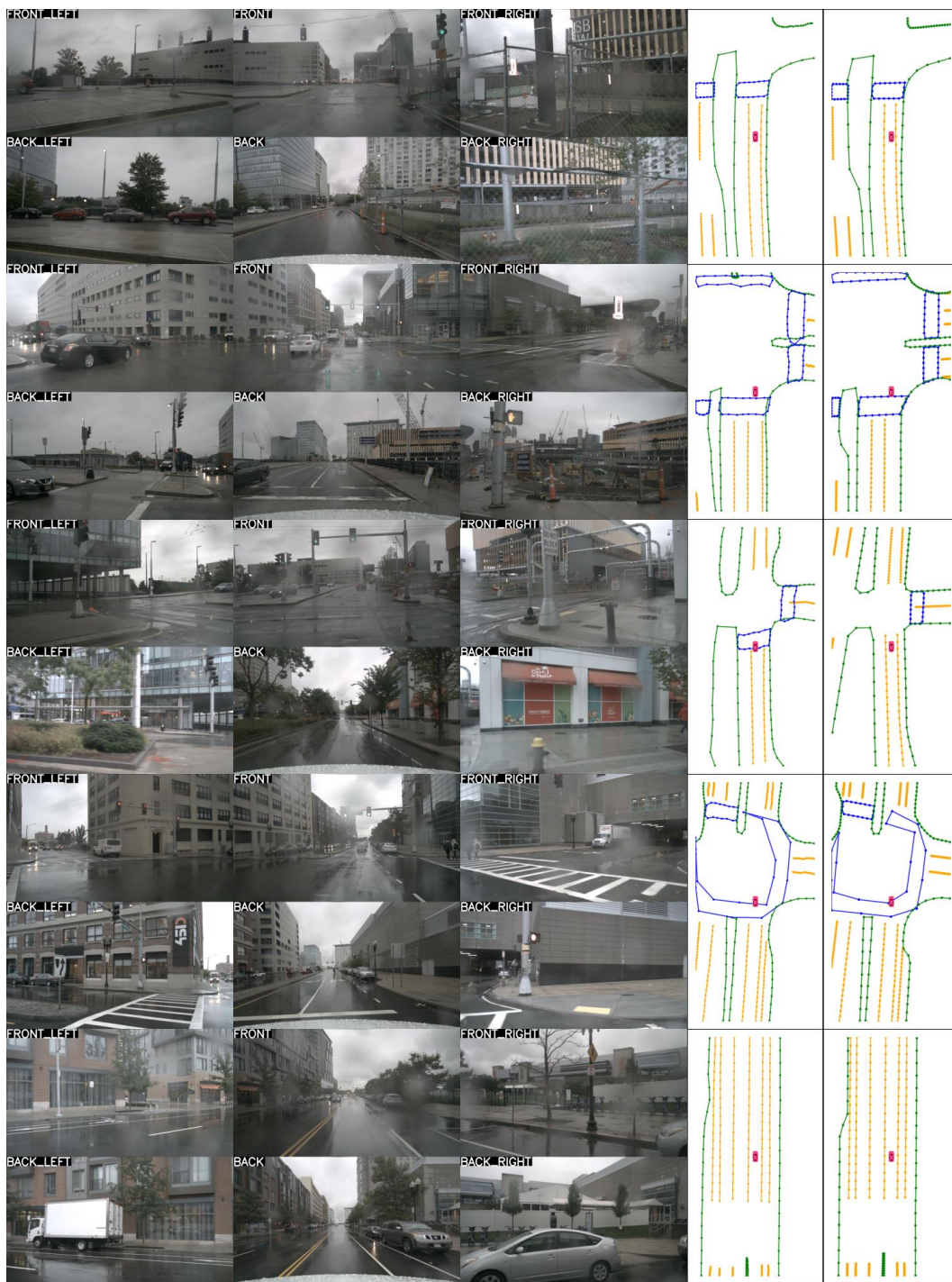


图 7. 雨天天气下的可视化。

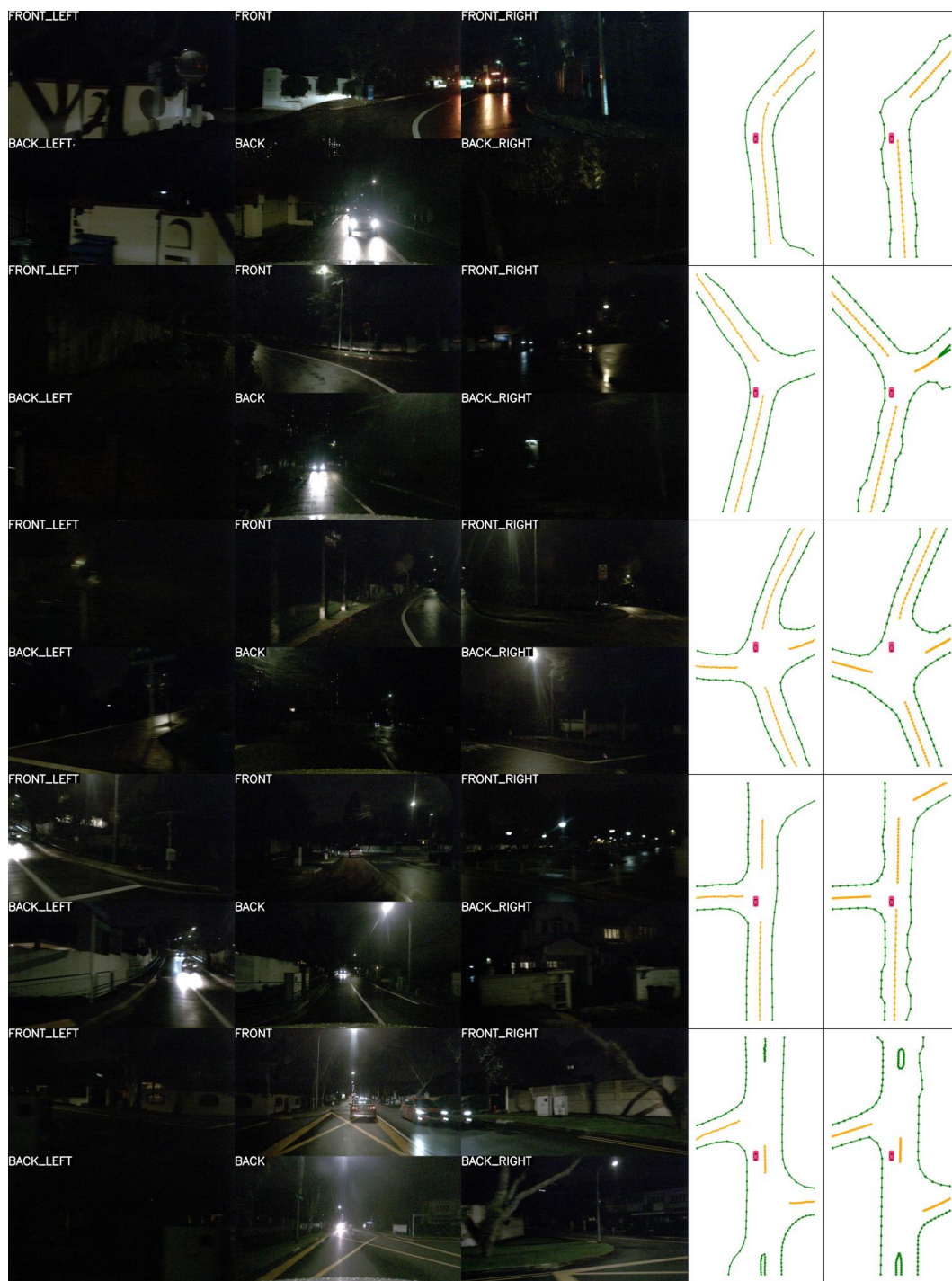


图 8. 夜晚条件下的可视化。